



UNIVERSITY COLLEGE OF ENGINEERING KANCHEEPURAM
APRIL/MAY 2026 EXAMINATIONS
COMPUTER SCIENCE AND ENGINEERING

**Agentic AI for Personal Task Planning and
Schedule Optimization**

BATCH MEMBERS:

- 1. ARUNKUMAR L (513422104704)**
- 2. MUKILAN S (513422104036)**
- 3. SUGANYA U (513422104042)**
- 4. AISHWARIYA D (513422104011)**

GUIDED BY:
DR .V. KAVITHA
PROFESSOR

INTRODUCTION

- ❖ **PROBLEM STATEMENT:** Today, people manage many daily tasks such as meetings, reminders, emails, and schedules using multiple apps. Managing everything manually is time-consuming and error-prone. Existing systems do not intelligently coordinate tasks or provide smart suggestions.
- ❖ **PROPOSED SOLUTION:** This project proposes an **AI-based multi-agent system** designed to aggregate data from Google services (Calendar, Gmail, Contacts, Sheets, Maps), reason over user context, and autonomously suggest actions and generate intelligent drafts, with future support for execution.
- ❖ **KEY TECHNOLOGIES USED:** Large Language Models (LLMs), Multi-Agent Systems, Task Planning and Coordination and Intelligent Automation.

ABSTRACT

1. Recent advancements in Large Language Models have made it possible for AI systems to understand natural language and assist users in real-life tasks.
2. This project focuses on building a **cooperative multi-agent system** where different agents work together to automate personal tasks such as scheduling, reminders, and planning activities.
3. Each agent has a specific role, such as understanding user requests, planning tasks, generating intelligent recommendations and explanations for personal task automation. By working together, the system provides smarter and more reliable task automation.
4. The proposed system aims to reduce human effort, improve productivity, and demonstrate how agent-based AI can be applied to everyday personal task management.

AIM & OBJECTIVE

AIM : To design and develop an AI-powered cooperative multi-agent system for automating personal daily tasks.

OBJECTIVES :

- ❖ To build multiple agents that work together to handle tasks
- ❖ To automate task analysis, planning, recommendation using cooperative AI agents.
- ❖ To reduce manual task management for users
- ❖ To improve task accuracy using agent collaboration

SCOPE

- ❖ Can be used for personal task management such as scheduling and reminders
- ❖ Suitable for students, working professionals, and individuals
- ❖ Can be extended to email automation, calendar management, and reminders
- ❖ Supports future improvements with advanced AI models
- ❖ Acts as a foundation for intelligent personal assistants
- ❖ **IMPACT:** The system reduces manual effort, improves task accuracy, and helps users manage daily activities efficiently using AI assistance.

EXISTING SYSTEM

- ❖ Users manually manage tasks using calendars and reminder apps.
- ❖ Traditional task management apps are rule-based
- ❖ Single AI systems handle tasks independently
- ❖ No intelligent coordination between multiple actions
- ❖ Limited automation and decision-making capability

EXISTING SYSTEM -DRAWBACKS

- ❖ **Lack of Intelligence:** No smart understanding of user intent
- ❖ **Manual Effort:** Users must enter and manage tasks manually
- ❖ **No Validation:** Errors are not automatically detected
- ❖ **Poor Coordination:** Tasks are not connected or optimized
- ❖ **Low Automation:** Requires frequent user interaction

LITERATURE REVIEW

S.NO	TITLE IEEE TRANSACTION	AUTHORS & YEAR	TECHNIQUE USED	DRAWBACKS
1	LLM-Based Agent Systems for Task Automation	Jinlong Wang , Yunting Wu - 2025	AI-driven task planning and execution	Mostly single-agent systems
2	Multi-Agent Reinforcement Learning for Task Offloading in Crowd-Edge Computing	Su Yao, Mu Wang, Ju Ren - 2025	Rule-based and AI-based automation	Limited adaptability

Literature Review				
S.No	Title IEEE Transaction	Authors & Year	Technique Used	Drawbacks
3	Multi-Agent Systems for Complex Problem Solving	Wooldridge, 2021	Agent collaboration and coordination frameworks	High coordination and communication overhead
4	AI-Based Task Scheduling	Kumar et al., 2022	Automated scheduling algorithms	Limited reasoning capability

PROPOSED SYSTEM

System Components:

- ❖ **Calendar Agent** – Fetches and analyzes schedule data (Calendar / simulated)
- ❖ **Email Agent** – Drafts email replies and follow-ups (Gmail – planned)
- ❖ **Contact Agent** – Identifies missed calls and contacts (planned)
- ❖ **Task & Finance Agent** – Manages income/expense data (Sheets – planned)
- ❖ **Navigation Agent** – Provides directions and travel context (Maps – planned)
- ❖ **Conflict & Planning Agent** – Optimizes schedules and actions
- ❖ **LLM Explanation Agent** – Generates human-readable responses

Working Principle:

- ❖ Current implementation demonstrates the core planning pipeline using simulated data.

PROPOSED SYSTEM ADVANTAGES

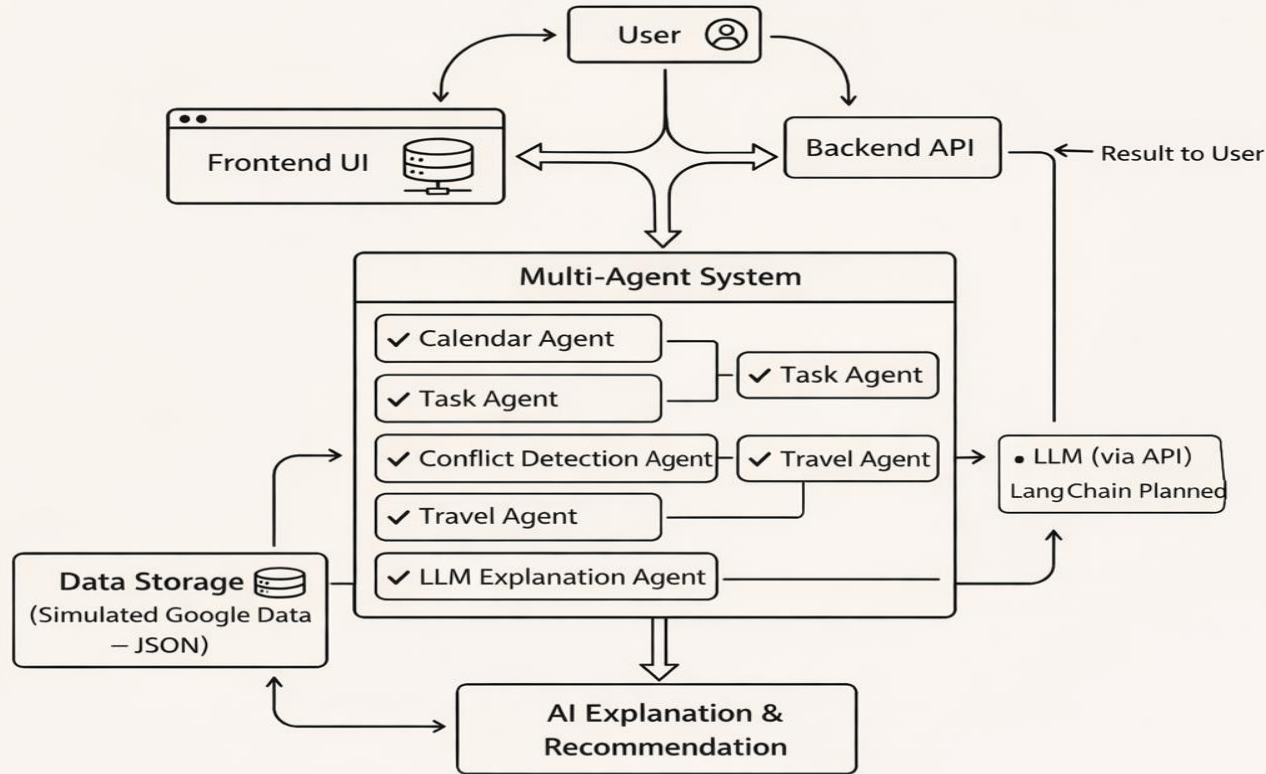
- ❖ Improved task accuracy
- ❖ Reduced manual effort
- ❖ Better task coordination
- ❖ Intelligent decision-making
- ❖ Scalable and flexible design
- ❖ User-friendly automation

SDLC PHASES

The Entire SDLC process is divided into the following stages:

- ❖ Requirement Collection and Analysis
- ❖ Feasibility Study
- ❖ System Design
- ❖ Implementation Planning
- ❖ Testing Strategy
- ❖ Deployment Planning
- ❖ Maintenance and Future Enhancements

ARCHITECTURAL DIAGRAM



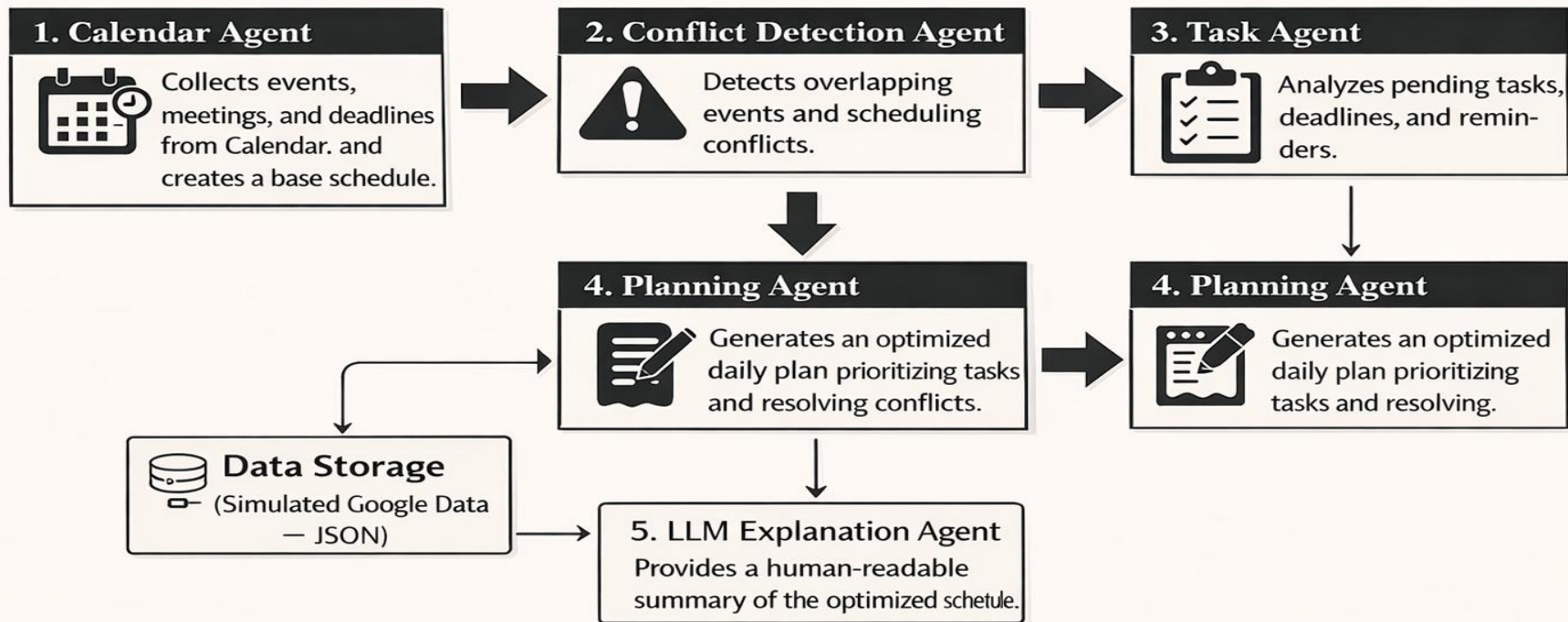
METHODOLOGY

- ❖ **Problem Analysis & Scope Definition** : Studied limitations of existing task management systems and defined scope for personal task automation using AI agents.
- ❖ **System Architecture Design** : Designed a multi-agent architecture where each agent performs a specific task and collaborates to complete user requests.
- ❖ **Agent Role Identification** : Defined roles for Planning Agent, Task Agent, Conflict Detection Agent, and Travel Reminder Agent.
- ❖ **Backend Development** : Developed backend using Python and FastAPI to handle user requests and agent communication.
- ❖ **LLM Integration** : Integrated LLM-based reasoning, with LangChain Planned to enable agents to understand tasks and make intelligent decisions.
- ❖ **Agent Implementation** : Implemented core agents and connected them into a cooperative automation pipeline.
- ❖ **Testing** : Tested API endpoints using sample meeting and tasks schedules.

APPLICATION EXAMPLE (POC)

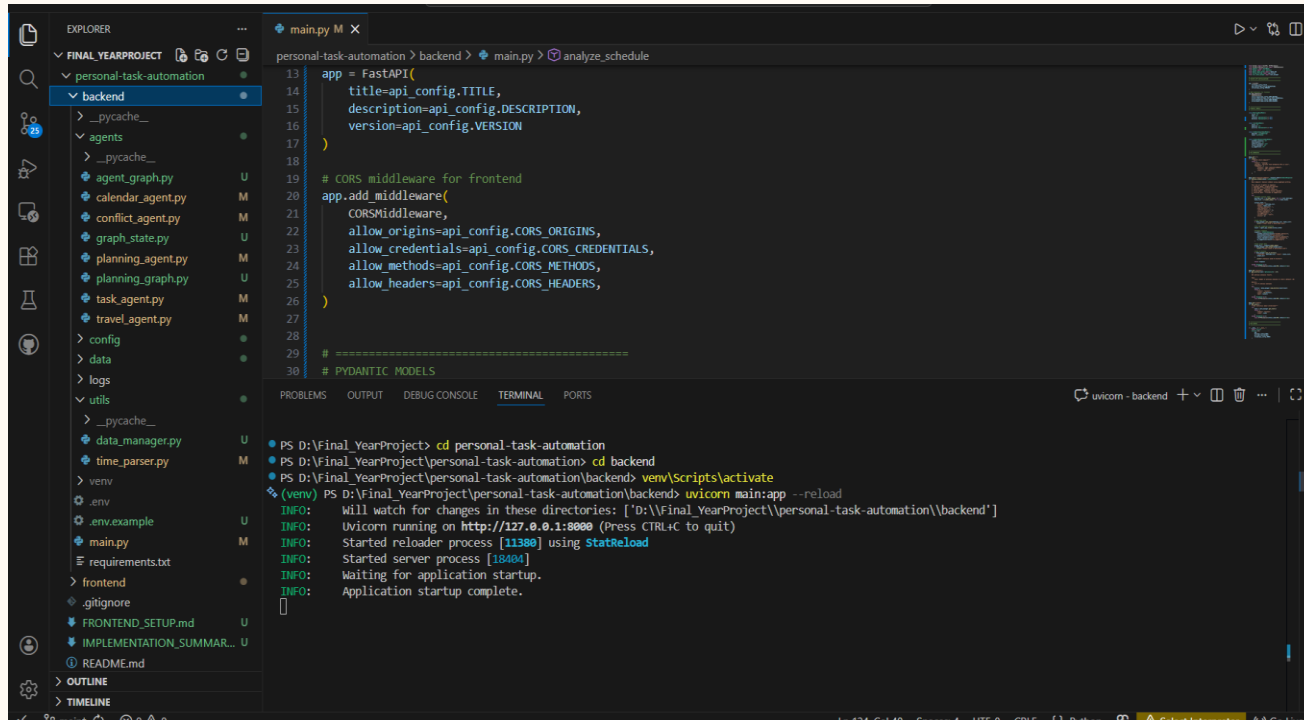
Use Case: Automated Personal Task Scheduler

User Request: “Automatically organize my schedule and tasks for the day.”



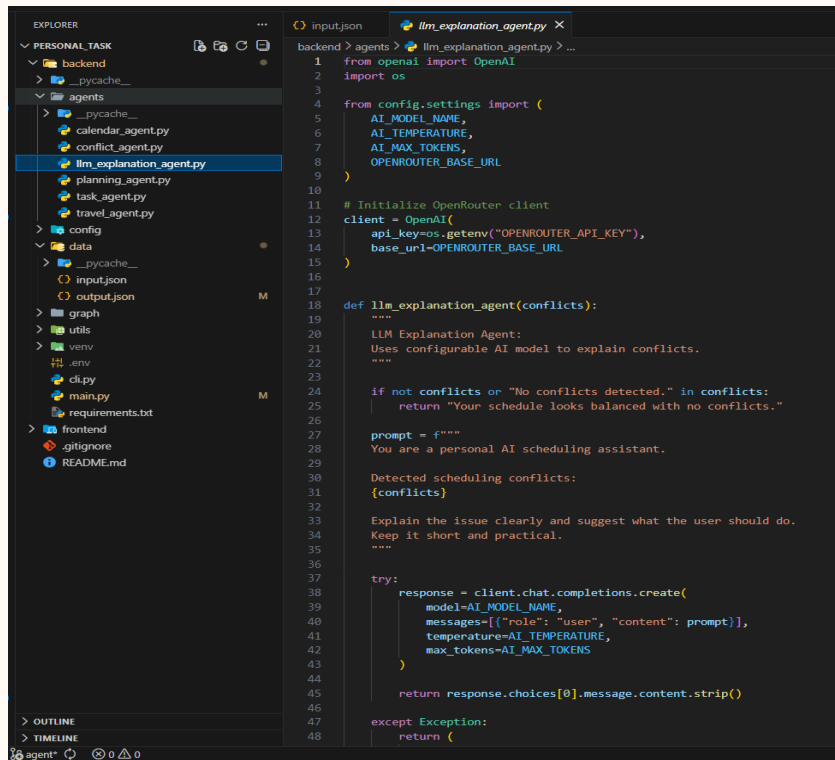
PROPOSED WORK

CODE BASES

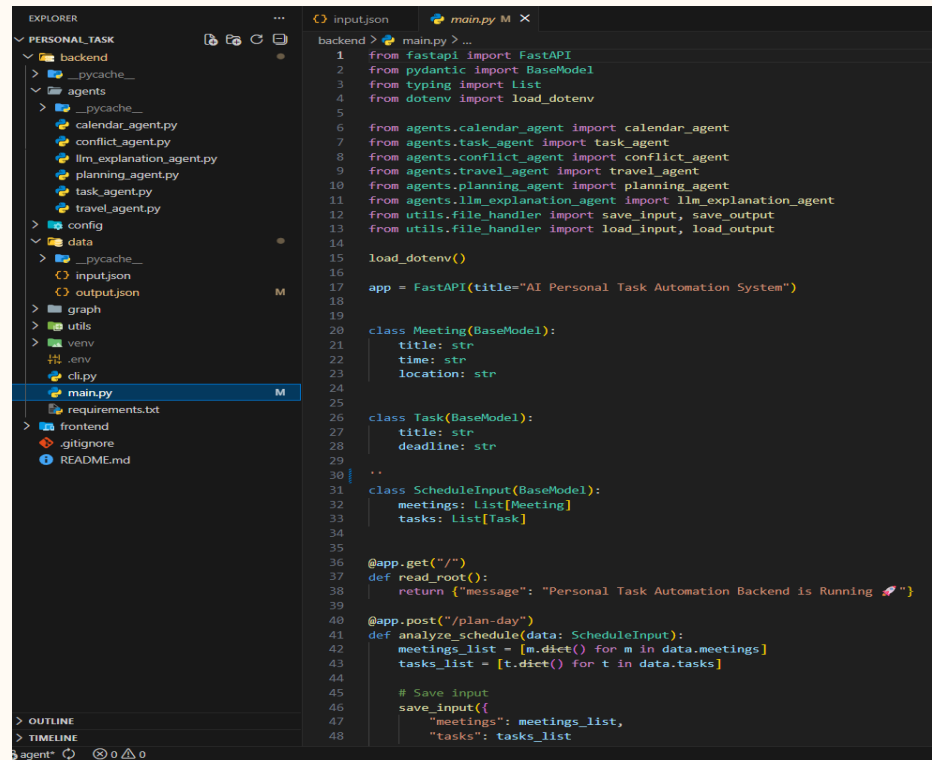


PROPOSED WORK

CODE BASES



```
1 from openai import OpenAI
2 import os
3
4 from config.settings import (
5     AI_MODEL_NAME,
6     AI_TEMPERATURE,
7     AI_MAX_TOKENS,
8     OPENROUTER_BASE_URL
9 )
10
11 # Initialize OpenRouter client
12 client = OpenAI(
13     api_key=os.getenv("OPENROUTER_API_KEY"),
14     base_url=OPENROUTER_BASE_URL
15 )
16
17 def llm_explanation_agent(conflicts):
18     """
19     LLM Explanation Agent:
20     Uses configurable AI model to explain conflicts.
21     """
22
23     if not conflicts or "No conflicts detected." in conflicts:
24         return "Your schedule looks balanced with no conflicts."
25
26     prompt = f"""
27     You are a personal AI scheduling assistant.
28     Detected scheduling conflicts:
29     {conflicts}
30
31     Explain the issue clearly and suggest what the user should do.
32     Keep it short and practical.
33     """
34
35     try:
36         response = client.chat.completions.create(
37             model=AI_MODEL_NAME,
38             messages=[{"role": "user", "content": prompt}],
39             temperature=AI_TEMPERATURE,
40             max_tokens=AI_MAX_TOKENS
41         )
42
43         return response.choices[0].message.content.strip()
44
45     except Exception:
46         return "
```



```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 from typing import List
4 from dotenv import load_dotenv
5
6 from agents.calendar_agent import calendar_agent
7 from agents.task_agent import task_agent
8 from agents.conflict_agent import conflict_agent
9 from agents.travel_agent import travel_agent
10 from agents.planning_agent import planning_agent
11 from agents.llm_explanation_agent import llm_explanation_agent
12 from utils.file_handler import save_input, save_output
13 from utils.file_handler import load_input, load_output
14
15 load_dotenv()
16
17 app = FastAPI(title="AI Personal Task Automation System")
18
19
20 class Meeting(BaseModel):
21     title: str
22     time: str
23     location: str
24
25
26 class Task(BaseModel):
27     title: str
28     deadline: str
29
30
31 class ScheduleInput(BaseModel):
32     meetings: List[Meeting]
33     tasks: List[Task]
34
35
36 @app.get("/")
37 def read_root():
38     return {"message": "Personal Task Automation Backend is Running 🚀"}
39
40
41 @app.post("/plan-day")
42 def analyze_schedule(data: ScheduleInput):
43     meetings_list = [m.dict() for m in data.meetings]
44     tasks_list = [t.dict() for t in data.tasks]
45
46     # Save input
47     save_input({
48         "meetings": meetings_list,
49         "tasks": tasks_list
50     })
```

SAMPLE OUTPUT

Swagger UI

Request body required

application/json

Edit Value | Schema

```
{  "meetings": [    {"title": "Team meeting", "time": "9:00 AM"}  ],  "tasks": [    {"title": "Finish report", "deadline": "10:00 AM"}  ]}
```

Execute Clear

Request URL

http://127.0.0.1:8080/analyse-schedule

Server response

Code Details

200

Response body

```
{  "calendar_analysis": [    "Today's Schedule Overview",    "9:00 AM - Team meeting",    "Key Focus: Team meeting: It's a great opportunity to align with your teammates and share updates.",    "Quick Tips: Review any notes or updates you want to share during the meeting. Come prepared with any questions or topics you'd like to discuss to make the most of your time together. Enjoy your meeting!",    "Task Analysis",    "Today's Task Overview",    "Finish report - Due: 10:00 AM",    "Critical Tasks: Finish report: CRITICAL - Due within 2 hours.",    "Quick Tips: Pack the report into sections to tackle it more easily. Set a timer for focused work sessions to boost your productivity! You've got this! Let's make today a success!",    "Conflict Analysis",    "Conflict 1*: Task deadline at 10:00 AM is within 30 minutes of the team meeting at 9:00 AM. Priority: Medium",    "Conflict 2*: Back-to-back commitments with no break due to the team meeting at 9:00 AM and task deadline at 10:00 AM. Priority: High",    "Conflict 3*: Unrealistic time to complete tasks between meetings, as the task 'Finish report' has a deadline that overlaps with the meeting. Priority: High",    "Travel Reminders",    "Sure! Here's your travel reminder for the upcoming meeting: Team meeting at 9:00 AM. Location: Not specified (check your calendar for details). Leave by 9:00 AM (no travel time needed). Tip: Don't forget to bring your laptop and any necessary documents! If you need any more reminders or details, just let me know! Safe travels!",    "Suggestions",    "Schedule Analysis: There are some conflicts in your schedule that could impact productivity, particularly with a team meeting at 9:00 AM and a task deadline at 10:00 AM, leading to back-to-back commitments without breaks.",    "Recommendations",    "Reschedule the Team Meeting: Move it to 10:30 AM or 11:00 AM for a better buffer before the task deadline.",    "Adjust the Task Deadline: Negotiate to extend the 'Finish report' deadline to 1:00 PM to avoid overlap with the meeting.",    "Incorporate Breaks: Add a short 5-10 minute break between the meeting and the task deadline to help team members regroup.",    "Action Items",    "Propose a new time for the team meeting.",    "Discuss the possibility of extending the current deadline.",    "Plan to implement a break in the schedule."  ],  "download": true}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: http://127.0.0.1:8080
content-length: 2563
content-type: application/json
date: Sun, 01 Feb 2026 14:06:58 GMT
server: uicorn
vary: Origin
```

CLI UI

```
PS C:\Users\arunk\Downloads\Final-Year Project\Personal_Task\backend> python cli.py
```

Calendar Analysis:
Meetings: Client Call at 9:00 am in Chennai, Team Standup at 10:30 AM in Office, Project Review at 12:00pm in Chennai, Design Discussion at 3:00 PM in Remote, Daily Wrap-up at 6:00 in Office

Task Analysis:
Tasks: Check Recent Project Updates due by 11:00 Pm, Prepare Client Presentation due by 10:15 A M, Fix Critical Bugs due by 2:30pm, Update Documentation due by 5:30 PM

Conflict Analysis:
- Conflict between 'Team Standup' and task 'Prepare Client Presentation'
- Conflict between 'Design Discussion' and task 'Fix Critical Bugs'

Travel Reminders:
- Leave for 'Client Call' by 08:30 AM
- Leave for 'Team Standup' by 10:00 AM
- Leave for 'Project Review' by 11:30 AM
- Leave for 'Design Discussion' by 02:30 PM
- Leave for 'Daily Wrap-up' by 05:30 AM

Rule-Based Plan:
- Suggestion: Reschedule the task or adjust meeting time to avoid conflict.
- Suggestion: Reschedule the task or adjust meeting time to avoid conflict.

AI Explanation:
Here's a clear explanation and practical steps for your scheduling conflicts:

The Issue:
You have two overlapping commitments:
1. "Team Standup" conflicts with "Prepare Client Presentation".
2. "Design Discussion" conflicts with "Fix Critical Bugs".

What to Do:
1. **Prioritize Urgency & Impact:**
* "Fix Critical Bugs" is likely urgent. Prioritize this over the "Design Discussion".
* "Prepare Client Presentation" is likely important. Compare it to the value of the "Team Standup" (which might be essential for daily coordination).

2. **Take Action:**
*

=== End of Analysis ===

REFERENCES

1. Wooldridge, M., *An Introduction to Multi-Agent Systems*, 2nd Edition, Wiley, 2021.
2. Russell, S., Norvig, P., *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson Education, 2021.
3. Brown, T. B., et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
4. OpenAI, “GPT-4 Technical Report,” 2023.
5. Zheng, C., et al., “Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems,” 2024.



THANK YOU